

Vehicle Parking App - V1

Project Report

Author:

Name: Johana Priscy John Ponraj

Roll Number: 23F2001738

Email: 23f2001738@ds.study.iitm.ac.in

I am a Data Science student at IIT Madras, enthusiastic about full-stack development and solving real-world problems through technology. This project honed my skills in building a practical web application for parking management.

Description

The Vehicle Parking App is a Flask-based web application for managing 4-wheeler parking lots and spots. It supports admin and user roles, enabling admins to manage lots and monitor usage, while users can register, reserve spots, and view parking history. The app features a responsive UI and analytics via summary charts.

AI/LLM Used

AI/LLM tools were used minimally (~5%) for debugging and code optimization suggestions. All core logic, design, and implementation were developed independently to meet project requirements.

Technologies Used

- **Flask:** Backend framework for routing and request handling.
- **Flask-SQLAlchemy:** ORM for SQLite database management.
- **Jinja2:** Templating engine for dynamic HTML rendering.
- **HTML/CSS/Bootstrap:** Responsive and user-friendly front-end design.
- **Chart.js:** Visualization of summary charts for admins and users.
- **SQLite:** Lightweight database for storing parking and user data.

Purpose: Flask and Flask-SQLAlchemy enable efficient backend and database operations. Bootstrap provides a responsive UI, and Chart.js supports data visualization for analytics.

DB Schema Design

The database comprises four tables:

1. ParkingLot:

- **Columns:** id (Integer, Primary Key), prime_location_name (String, Not Null), price (Float, Not Null), address (String, Not Null), pin_code (String, Not Null), maximum_number_of_spots (Integer, Not Null).
- **Constraints:** All fields are mandatory to ensure complete lot information.

2. ParkingSpot:

- **Columns:** id (Integer, Primary Key), lot_id (Integer, Foreign Key to ParkingLot, Not Null), status (String, Not Null, Default 'A').
- **Constraints:** Foreign key links to ParkingLot; status defaults to 'A' (Available).

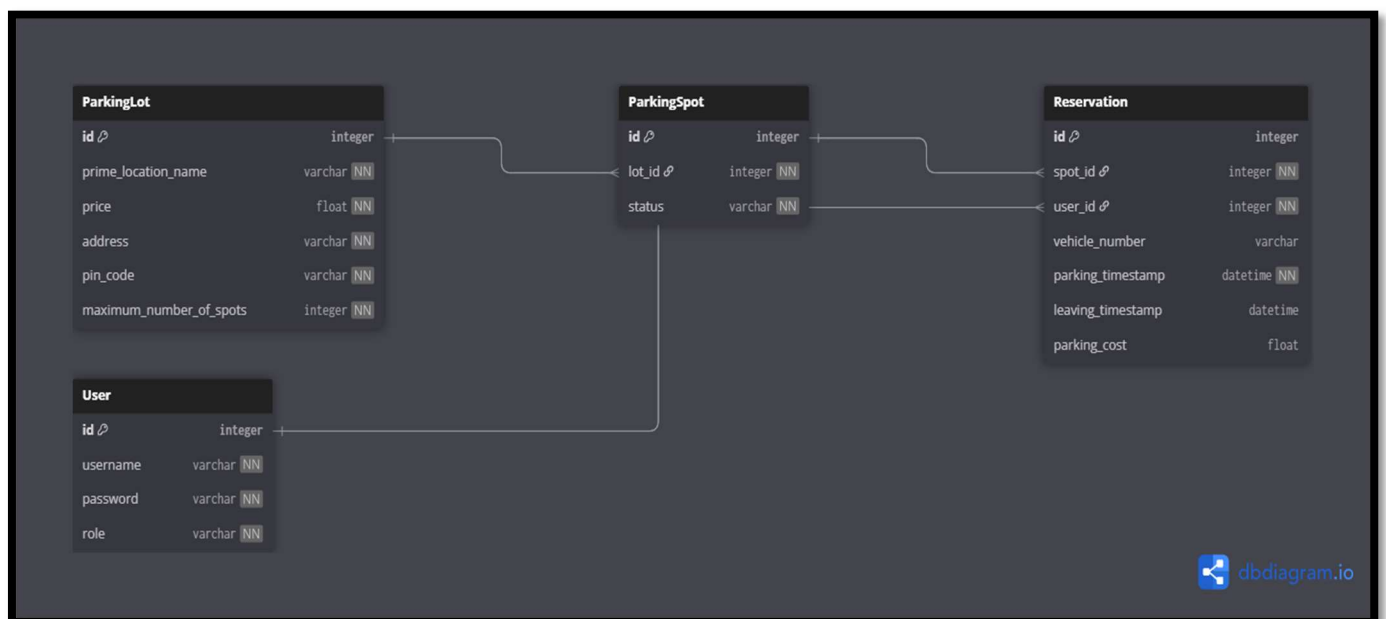
3. User:

- **Columns:** id (Integer, Primary Key), username (String, Unique, Not Null), password (String, Not Null), role (String, Not Null, Default 'user').
- **Constraints:** Unique username prevents duplicates; role differentiates admin/user.

4. Reservation:

- **Columns:** id (Integer, Primary Key), spot_id (Integer, Foreign Key to ParkingSpot, Not Null), user_id (Integer, Foreign Key to User, Not Null), vehicle_number (String, Nullable), parking_timestamp (DateTime, Not Null), leaving_timestamp (DateTime, Nullable), parking_cost (Float, Nullable).
- **Constraints:** Foreign keys ensure relational integrity; nullable fields support active reservations.

Rationale: Foreign keys maintain data relationships, and nullable fields accommodate ongoing reservations. Defaults and constraints ensure data consistency and simplify management.



API Design

The API includes one endpoint: `/api/spots` (GET), which returns a JSON array of parking spots with `id`, `lot_id`, and `status`. Implemented using Flask's `jsonify`, it supports external system integration. The API is documented in `api.yaml` using OpenAPI 3.0.0, defining the endpoint's structure and responses for clarity.

Architecture and Features

Structure:

parking_app_23f2001738/

```
├── api.yaml
├── app.py
├── models.py
├── README.md
├── requirements.txt
├── instance/
│   └── parking.db
├── static/
│   └── css/
│       └── style.css
├── templates/
│   ├── admin_dashboard.html
│   ├── base.html
│   ├── edit_lot.html
│   ├── login.html
│   ├── register.html
│   ├── spot_details.html
│   ├── user_dashboard.html
│   └── view_spot.html
```

The project is organized with `app.py` as the main Flask application, containing routes and controllers. Database models are defined in `models.py` using Flask-SQLAlchemy. Templates (`base.html`, `admin_dashboard.html`, `user_dashboard.html`, etc.) are in the `templates` folder, extending a common layout with Jinja2. Static assets (Bootstrap, Chart.js) are stored in the `static` folder.

Core Features:

- **Admin Dashboard:** Auto-created admin (username: admin, password: admin123) can create, edit, or delete parking lots (if all spots are available). Admins view spot statuses, reservation details (user, vehicle, timestamps), and user data. Summary charts display total reservations, spots, and revenue per lot using Chart.js.
- **User Dashboard:** Users register, log in, reserve spots (auto-allocated from available spots), and release them, with costs calculated based on parking duration and lot price. Users view their reservation history and summary charts.
- **Additional Features:** Search functionality for lots by name, HTML5 form validation, and a responsive Bootstrap-based UI. Chart.js visualizes total spots and reservations for both roles.

Video:

<https://drive.google.com/file/d/1UktasJdO83wZW3pxuTEsT8lU73d6ZYrj/view?usp=sharing>